

Example: Classification of Movie Reviews with a Fully Connected Neural Network Using Bag of Words

This notebook presents an example of text classification. The code can be found in Chapter 3, Section 5 of [Deep Learning with Python](#) by Francois Chollet.

```
In [50]: import keras
```

```
In [ ]:
```

Classifying movie reviews: a binary classification example

This notebook contains the code samples found in Chapter 3, Section 5 of [Deep Learning with Python](#). Note that the original text features far more content, in particular further explanations and figures: in this notebook, you will only find source code and related comments.

Two-class classification, or binary classification, may be the most widely applied kind of machine learning problem. In this example, we will learn to classify movie reviews into "positive" reviews and "negative" reviews, just based on the text content of the reviews.

The IMDB dataset

We'll be working with "IMDB dataset", a set of 50,000 highly-polarized reviews from the Internet Movie Database. They are split into 25,000 reviews for training and 25,000 reviews for testing, each set consisting in 50% negative and 50% positive reviews.

Why do we have these two separate training and test sets? You should never test a machine learning model on the same data that you used to train it! Just because a model performs well on its training data doesn't mean that it will perform well on data it has never seen, and what you actually care about is your model's performance on new data (since you already know the labels of your training data -- obviously you don't need your model to predict those). For instance, it is possible that your model could end up merely *memorizing* a mapping between your training samples and their targets -- which would be completely useless for the task of predicting targets for data never seen before. We will go over this point in much more detail in the next chapter.

Just like the MNIST dataset, the IMDB dataset comes packaged with Keras. It has already been preprocessed: the reviews (sequences of words) have been turned into sequences of integers, where each integer stands for a specific word in a dictionary.

The following code will load the dataset (when you run it for the first time, about 80MB of data will be downloaded to your machine):

```
In [51]: from keras.datasets import imdb  
  
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(num_words=
```

The argument `num_words=10000` means that we will only keep the top 10,000 most frequently occurring words in the training data. Rare words will be discarded. This allows us to work with vector data of manageable size.

The variables `train_data` and `test_data` are lists of reviews, each review being a list of word indices (encoding a sequence of words). `train_labels` and `test_labels` are lists of 0s and 1s, where 0 stands for "negative" and 1 stands for "positive":

```
In [52]: train_data[0]
```

```
Out[52]: [1,
          14,
          22,
          16,
          43,
          530,
          973,
          1622,
          1385,
          65,
          458,
          4468,
          66,
          3941,
          4,
          173,
          36,
          256,
          5,
          25,
          100,
          43,
          838,
          112,
          50,
          670,
          2,
          9,
          35,
          480,
          284,
          5,
          150,
          4,
          172,
          112,
          167,
          2,
          336,
          385,
          39,
          4,
          172,
          4536,
          1111,
          17,
          546,
          38,
          13,
          447,
          4,
          192,
          50,
          16,
          6,
          147,
          2025,
          19,
          14,
          22,
```

4,
1920,
4613,
469,
4,
22,
71,
87,
12,
16,
43,
530,
38,
76,
15,
13,
1247,
4,
22,
17,
515,
17,
12,
16,
626,
18,
2,
5,
62,
386,
12,
8,
316,
8,
106,
5,
4,
2223,
5244,
16,
480,
66,
3785,
33,
4,
130,
12,
16,
38,
619,
5,
25,
124,
51,
36,
135,
48,
25,
1415,
33,

6,
22,
12,
215,
28,
77,
52,
5,
14,
407,
16,
82,
2,
8,
4,
107,
117,
5952,
15,
256,
4,
2,
7,
3766,
5,
723,
36,
71,
43,
530,
476,
26,
400,
317,
46,
7,
4,
2,
1029,
13,
104,
88,
4,
381,
15,
297,
98,
32,
2071,
56,
26,
141,
6,
194,
7486,
18,
4,
226,
22,
21,

```
134,  
476,  
26,  
480,  
5,  
144,  
30,  
5535,  
18,  
51,  
36,  
28,  
224,  
92,  
25,  
104,  
4,  
226,  
65,  
16,  
38,  
1334,  
88,  
12,  
16,  
283,  
5,  
16,  
4472,  
113,  
103,  
32,  
15,  
16,  
5345,  
19,  
178,  
32]
```

```
In [53]: train_labels[0]
```

```
Out[53]: np.int64(1)
```

Since we restricted ourselves to the top 10,000 most frequent words, no word index will exceed 10,000:

```
In [54]: max([max(sequence) for sequence in train_data])
```

```
Out[54]: 9999
```

For kicks, here's how you can quickly decode one of these reviews back to English words:

```
In [55]: # word_index is a dictionary mapping words to an integer index  
word_index = imdb.get_word_index()  
# We reverse it, mapping integer indices to words  
reverse_word_index = dict([(value, key) for (key, value) in word_index.items()])  
# We decode the review; note that our indices were offset by 3
```

```
# because 0, 1 and 2 are reserved indices for "padding", "start of sequence", and
decoded_review = ' '.join([reverse_word_index.get(i - 3, '?') for i in train_data
```

In [56]: decoded_review

Out[56]: "? this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert ? is an amazing actor and now the same being director ? father came from the same scottish island as myself so i loved the fact there was a real connection with this film the witty remarks throughout the film were great it was just brilliant so much that i bought the film as soon as it was released for ? and would recommend it to everyone to watch and the fly fishing was amazing really cried at the end it was so sad and you know what they say if you cry at a film it must have been good and this definitely was also ? to the two little boy's that played the ? of norman and paul they were just brilliant children are often left out of the ? list i think because the stars that play them all grown up are such a big profile for the whole film but these children are amazing and should be praised for what they have done don't you think the whole story was so lovely because it was true and was someone's life after all that was shared with us all"

Preparing the data

We cannot feed lists of integers into a neural network. We have to turn our lists into tensors. There are two ways we could do that:

- We could pad our lists so that they all have the same length, and turn them into an integer tensor of shape `(samples, word_indices)`,

then use as first layer in our network a layer capable of handling such integer tensors (the `Embedding` layer, which we will cover in detail later in the book).

- We could one-hot-encode our lists to turn them into vectors of 0s and 1s. Concretely, this would mean for instance turning the sequence

`[3, 5]` into a 10,000-dimensional vector that would be all-zeros except for indices 3 and 5, which would be ones. Then we could use as first layer in our network a `Dense` layer, capable of handling floating point vector data.

We will go with the latter solution. Let's vectorize our data, which we will do manually for maximum clarity:

```
In [57]: import numpy as np

def vectorize_sequences(sequences, dimension=10000):
    # Create an all-zero matrix of shape (len(sequences), dimension)
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1. # set specific indices of results[i] to 1s
    return results

# Our vectorized training data
x_train = vectorize_sequences(train_data)
# Our vectorized test data
x_test = vectorize_sequences(test_data)
```

Here's what our samples look like now:

```
In [58]: x_train[0]
```

```
Out[58]: array([0., 1., 1., ..., 0., 0., 0.], shape=(10000,))
```

We should also vectorize our labels, which is straightforward:

```
In [59]: # Our vectorized labels
y_train = np.asarray(train_labels).astype('float32')
y_test = np.asarray(test_labels).astype('float32')
```

Now our data is ready to be fed into a neural network.

Building our network

Our input data is simply vectors, and our labels are scalars (1s and 0s): this is the easiest setup you will ever encounter. A type of network that performs well on such a problem would be a simple stack of fully-connected (`Dense`) layers with `relu` activations:

```
Dense(16, activation='relu')
```

The argument being passed to each `Dense` layer (16) is the number of "hidden units" of the layer. What's a hidden unit? It's a dimension in the representation space of the layer. You may remember from the previous chapter that each such `Dense` layer with a `relu` activation implements the following chain of tensor operations:

```
output = relu(dot(W, input) + b)
```

Having 16 hidden units means that the weight matrix `W` will have shape `(input_dimension, 16)`, i.e. the dot product with `W` will project the input data onto a 16-dimensional representation space (and then we would add the bias vector `b` and apply the `relu` operation). You can intuitively understand the dimensionality of your representation space as "how much freedom you are allowing the network to have when learning internal representations". Having more hidden units (a higher-dimensional representation space) allows your network to learn more complex representations, but it makes your network more computationally expensive and may lead to learning

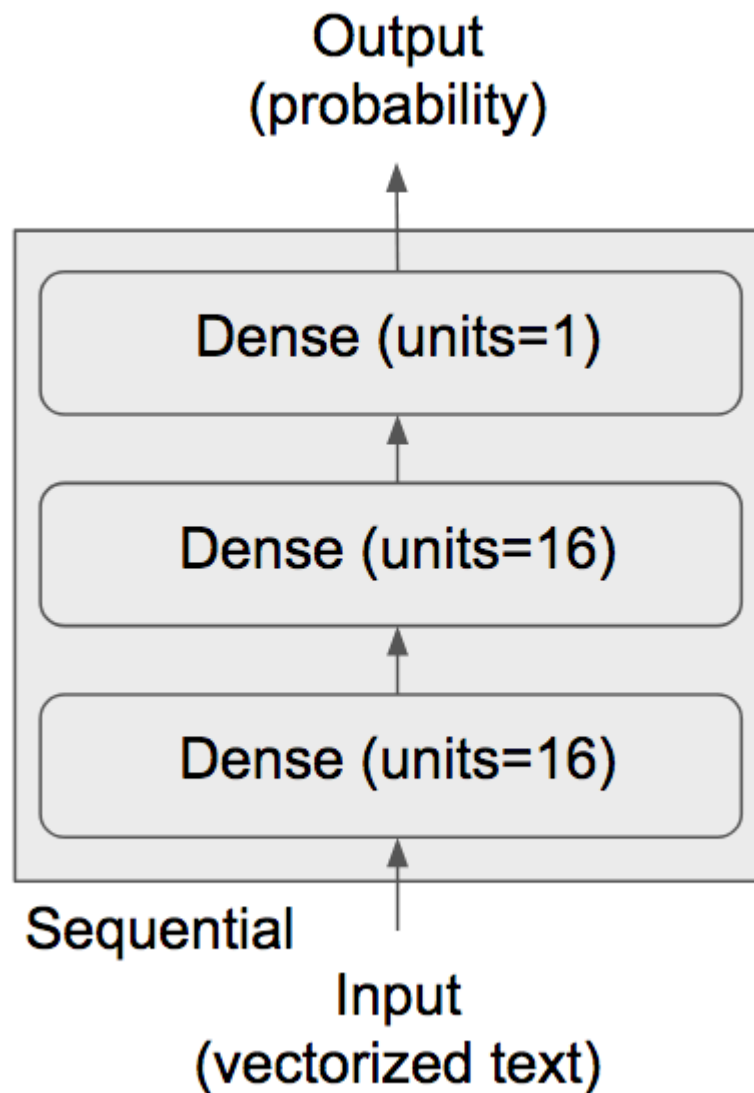
unwanted patterns (patterns that will improve performance on the training data but not on the test data).

There are two key architecture decisions to be made about such stack of dense layers:

- How many layers to use.
- How many "hidden units" to chose for each layer.

In the next chapter, you will learn formal principles to guide you in making these choices. For the time being, you will have to trust us with the following architecture choice: two intermediate layers with 16 hidden units each, and a third layer which will output the scalar prediction regarding the sentiment of the current review. The intermediate layers will use `relu` as their "activation function", and the final layer will use a sigmoid activation so as to output a probability (a score between 0 and 1, indicating how likely the sample is to have the target "1", i.e. how likely the review is to be positive). A `relu` (rectified linear unit) is a function meant to zero-out negative values, while a sigmoid "squashes" arbitrary values into the `[0, 1]` interval, thus outputting something that can be interpreted as a probability.

Here's what our network looks like:



And here's the Keras implementation, very similar to the MNIST example you saw previously:

```
In [60]: from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

Lastly, we need to pick a loss function and an optimizer. Since we are facing a binary classification problem and the output of our network is a probability (we end our network with a single-unit layer with a sigmoid activation), is it best to use the `binary_crossentropy` loss. It isn't the only viable choice: you could use, for instance, `mean_squared_error`. But crossentropy is usually the best choice when you are dealing with models that output probabilities. Crossentropy is a quantity from the field of

Information Theory, that measures the "distance" between probability distributions, or in our case, between the ground-truth distribution and our predictions.

Here's the step where we configure our model with the `rmsprop` optimizer and the `binary_crossentropy` loss function. Note that we will also monitor accuracy during training.

```
In [61]: model.compile(optimizer='rmsprop',  
                      loss='binary_crossentropy',  
                      metrics=['accuracy'])
```

We are passing our optimizer, loss function and metrics as strings, which is possible because `rmsprop`, `binary_crossentropy` and `accuracy` are packaged as part of Keras. Sometimes you may want to configure the parameters of your optimizer, or pass a custom loss function or metric function. This former can be done by passing an optimizer class instance as the `optimizer` argument:

```
In [62]: from keras import optimizers  
  
model.compile(optimizer=optimizers.RMSprop(learning_rate=0.001),  
              loss='binary_crossentropy',  
              metrics=['accuracy'])
```

The latter can be done by passing function objects as the `loss` or `metrics` arguments:

```
In [63]: from keras import losses  
         from keras import metrics  
  
model.compile(optimizer=optimizers.RMSprop(learning_rate=0.001),  
              loss=losses.binary_crossentropy,  
              metrics=[metrics.binary_accuracy])
```

Validating our approach

In order to monitor during training the accuracy of the model on data that it has never seen before, we will create a "validation set" by setting apart 10,000 samples from the original training data:

```
In [64]: x_val = x_train[:10000]  
         partial_x_train = x_train[10000:]  
  
         y_val = y_train[:10000]  
         partial_y_train = y_train[10000:]
```

We will now train our model for 20 epochs (20 iterations over all samples in the `x_train` and `y_train` tensors), in mini-batches of 512 samples. At this same time we will monitor loss and accuracy on the 10,000 samples that we set apart. This is done by passing the validation data as the `validation_data` argument:

```
In [65]: history = model.fit(partial_x_train,  
                             partial_y_train,  
                             epochs=40,  
                             batch_size=512,  
                             validation_data=(x_val, y_val)  
                             )
```

Epoch 1/40
30/30  1s 19ms/step - binary_accuracy: 0.7739 - loss: 0.5339
- val_binary_accuracy: 0.8387 - val_loss: 0.4196

Epoch 2/40
30/30  0s 6ms/step - binary_accuracy: 0.8929 - loss: 0.3284 -
val_binary_accuracy: 0.8793 - val_loss: 0.3210

Epoch 3/40
30/30  0s 6ms/step - binary_accuracy: 0.9192 - loss: 0.2427 -
val_binary_accuracy: 0.8839 - val_loss: 0.2920

Epoch 4/40
30/30  0s 6ms/step - binary_accuracy: 0.9367 - loss: 0.1932 -
val_binary_accuracy: 0.8901 - val_loss: 0.2747

Epoch 5/40
30/30  0s 6ms/step - binary_accuracy: 0.9480 - loss: 0.1578 -
val_binary_accuracy: 0.8900 - val_loss: 0.2773

Epoch 6/40
30/30  0s 8ms/step - binary_accuracy: 0.9559 - loss: 0.1354 -
val_binary_accuracy: 0.8881 - val_loss: 0.2855

Epoch 7/40
30/30  0s 7ms/step - binary_accuracy: 0.9639 - loss: 0.1165 -
val_binary_accuracy: 0.8877 - val_loss: 0.2983

Epoch 8/40
30/30  0s 7ms/step - binary_accuracy: 0.9702 - loss: 0.0981 -
val_binary_accuracy: 0.8846 - val_loss: 0.3116

Epoch 9/40
30/30  0s 7ms/step - binary_accuracy: 0.9777 - loss: 0.0805 -
val_binary_accuracy: 0.8822 - val_loss: 0.3286

Epoch 10/40
30/30  0s 6ms/step - binary_accuracy: 0.9830 - loss: 0.0676 -
val_binary_accuracy: 0.8738 - val_loss: 0.3618

Epoch 11/40
30/30  0s 6ms/step - binary_accuracy: 0.9858 - loss: 0.0590 -
val_binary_accuracy: 0.8610 - val_loss: 0.4664

Epoch 12/40
30/30  0s 6ms/step - binary_accuracy: 0.9899 - loss: 0.0478 -
val_binary_accuracy: 0.8716 - val_loss: 0.4391

Epoch 13/40
30/30  0s 6ms/step - binary_accuracy: 0.9925 - loss: 0.0399 -
val_binary_accuracy: 0.8738 - val_loss: 0.4119

Epoch 14/40
30/30  0s 6ms/step - binary_accuracy: 0.9918 - loss: 0.0349 -
val_binary_accuracy: 0.8763 - val_loss: 0.4388

Epoch 15/40
30/30  0s 6ms/step - binary_accuracy: 0.9941 - loss: 0.0291 -
val_binary_accuracy: 0.8743 - val_loss: 0.4505

Epoch 16/40
30/30  0s 6ms/step - binary_accuracy: 0.9983 - loss: 0.0192 -
val_binary_accuracy: 0.8734 - val_loss: 0.4784

Epoch 17/40
30/30  0s 6ms/step - binary_accuracy: 0.9977 - loss: 0.0181 -
val_binary_accuracy: 0.8643 - val_loss: 0.5250

Epoch 18/40
30/30  0s 6ms/step - binary_accuracy: 0.9967 - loss: 0.0180 -
val_binary_accuracy: 0.8734 - val_loss: 0.5228

Epoch 19/40
30/30  0s 6ms/step - binary_accuracy: 0.9994 - loss: 0.0103 -
val_binary_accuracy: 0.8709 - val_loss: 0.5494

Epoch 20/40
30/30  0s 7ms/step - binary_accuracy: 0.9986 - loss: 0.0111 -
val_binary_accuracy: 0.8699 - val_loss: 0.6097

Epoch 21/40
30/30  0s 6ms/step - binary_accuracy: 0.9998 - loss: 0.0063 -
val_binary_accuracy: 0.8722 - val_loss: 0.5930

Epoch 22/40
30/30  0s 6ms/step - binary_accuracy: 0.9973 - loss: 0.0122 -
val_binary_accuracy: 0.8718 - val_loss: 0.6096

Epoch 23/40
30/30  0s 6ms/step - binary_accuracy: 0.9999 - loss: 0.0043 -
val_binary_accuracy: 0.8691 - val_loss: 0.6327

Epoch 24/40
30/30  0s 7ms/step - binary_accuracy: 0.9978 - loss: 0.0097 -
val_binary_accuracy: 0.8701 - val_loss: 0.6464

Epoch 25/40
30/30  0s 6ms/step - binary_accuracy: 0.9999 - loss: 0.0031 -
val_binary_accuracy: 0.8723 - val_loss: 0.6622

Epoch 26/40
30/30  0s 6ms/step - binary_accuracy: 0.9979 - loss: 0.0083 -
val_binary_accuracy: 0.8717 - val_loss: 0.6835

Epoch 27/40
30/30  0s 6ms/step - binary_accuracy: 0.9999 - loss: 0.0023 -
val_binary_accuracy: 0.8711 - val_loss: 0.6975

Epoch 28/40
30/30  0s 6ms/step - binary_accuracy: 0.9978 - loss: 0.0083 -
val_binary_accuracy: 0.8654 - val_loss: 0.7462

Epoch 29/40
30/30  0s 6ms/step - binary_accuracy: 0.9999 - loss: 0.0018 -
val_binary_accuracy: 0.8704 - val_loss: 0.7296

Epoch 30/40
30/30  0s 7ms/step - binary_accuracy: 1.0000 - loss: 0.0017 -
val_binary_accuracy: 0.8665 - val_loss: 0.7578

Epoch 31/40
30/30  0s 6ms/step - binary_accuracy: 0.9985 - loss: 0.0054 -
val_binary_accuracy: 0.8676 - val_loss: 0.7679

Epoch 32/40
30/30  0s 6ms/step - binary_accuracy: 1.0000 - loss: 0.0012 -
val_binary_accuracy: 0.8695 - val_loss: 0.7739

Epoch 33/40
30/30  0s 6ms/step - binary_accuracy: 1.0000 - loss: 0.0010 -
val_binary_accuracy: 0.8647 - val_loss: 0.8125

Epoch 34/40
30/30  0s 7ms/step - binary_accuracy: 0.9991 - loss: 0.0039 -
val_binary_accuracy: 0.8698 - val_loss: 0.8210

Epoch 35/40
30/30  0s 6ms/step - binary_accuracy: 1.0000 - loss: 7.9515e-04 -
val_binary_accuracy: 0.8700 - val_loss: 0.8276

Epoch 36/40
30/30  0s 7ms/step - binary_accuracy: 1.0000 - loss: 6.8801e-04 -
val_binary_accuracy: 0.8701 - val_loss: 0.8448

Epoch 37/40
30/30  0s 6ms/step - binary_accuracy: 0.9984 - loss: 0.0059 -
val_binary_accuracy: 0.8705 - val_loss: 0.8572

Epoch 38/40
30/30  0s 6ms/step - binary_accuracy: 1.0000 - loss: 5.8188e-04 -
val_binary_accuracy: 0.8703 - val_loss: 0.8648

Epoch 39/40
30/30  0s 6ms/step - binary_accuracy: 1.0000 - loss: 5.1344e-04 -
val_binary_accuracy: 0.8688 - val_loss: 0.8822

Epoch 40/40
30/30  0s 6ms/step - binary_accuracy: 1.0000 - loss: 4.4149e-04 -
val_binary_accuracy: 0.8713 - val_loss: 0.9193

On CPU, this will take less than two seconds per epoch -- training is over in 20 seconds. At the end of every epoch, there is a slight pause as the model computes its loss and accuracy on the 10,000 samples of the validation data.

Note that the call to `model.fit()` returns a `History` object. This object has a member `history`, which is a dictionary containing data about everything that happened during training. Let's take a look at it:

```
In [66]: history_dict = history.history
history_dict.keys()
```

```
Out[66]: dict_keys(['binary_accuracy', 'loss', 'val_binary_accuracy', 'val_loss'])
```

It contains 4 entries: one per metric that was being monitored, during training and during validation. Let's use Matplotlib to plot the training and validation loss side by side, as well as the training and validation accuracy:

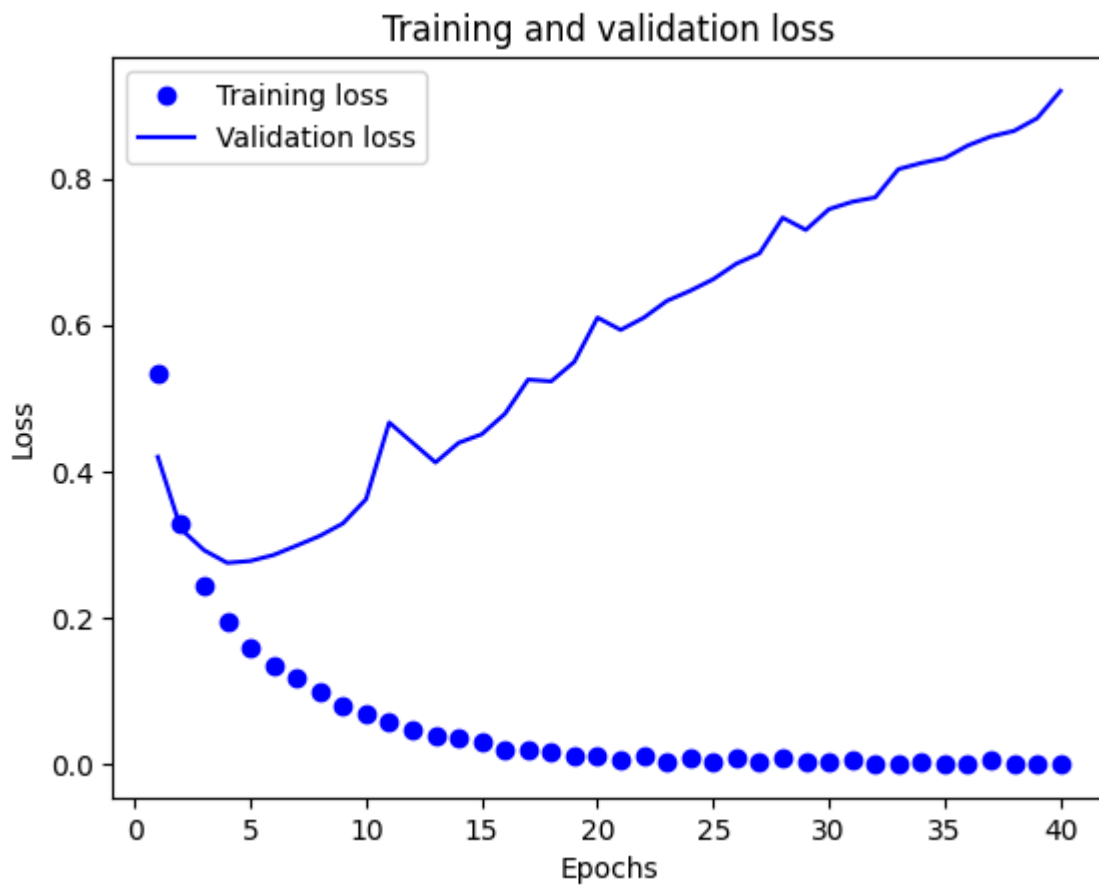
```
In [68]: import matplotlib.pyplot as plt

acc = history.history['binary_accuracy']
val_acc = history.history['val_binary_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

# "bo" is for "blue dot"
plt.plot(epochs, loss, 'bo', label='Training loss')
# b is for "solid blue line"
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

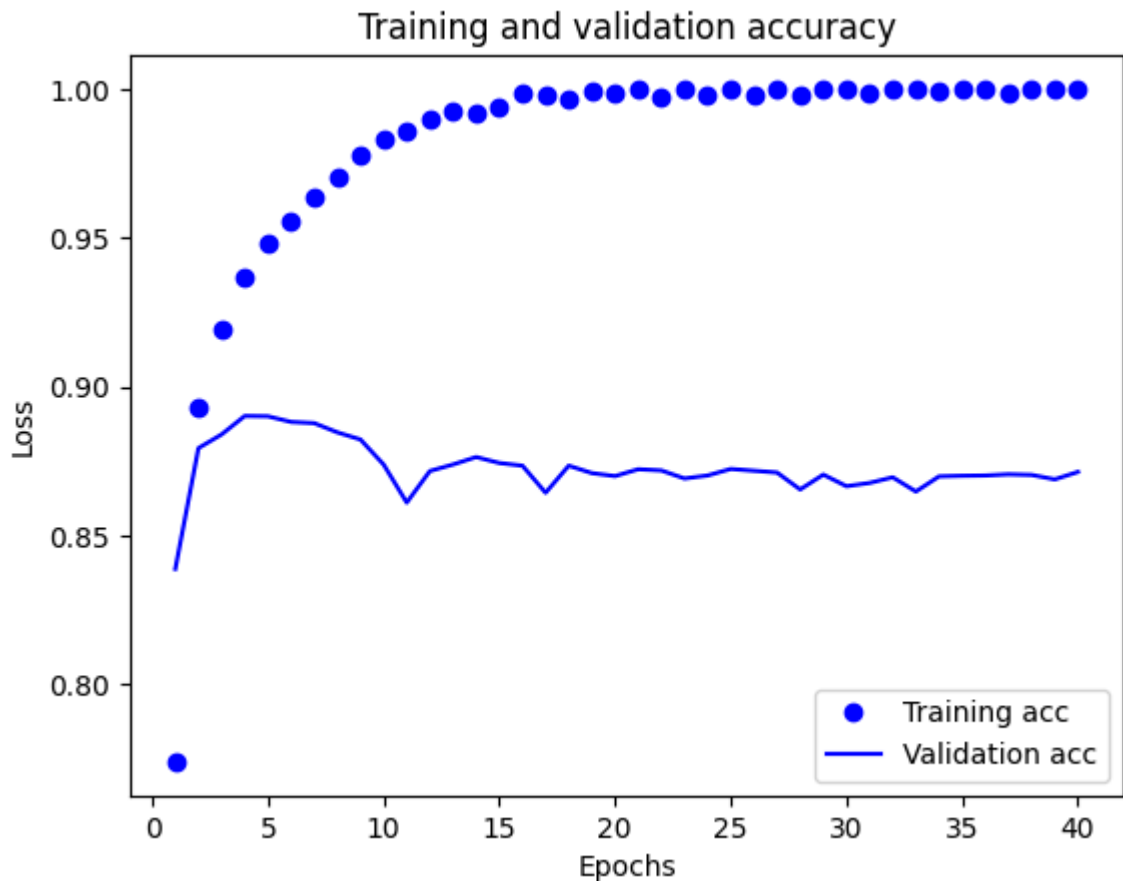
plt.show()
```



```
In [69]: plt.clf() # clear figure
acc_values = history_dict['binary_accuracy']
val_acc_values = history_dict['val_binary_accuracy']

plt.plot(epochs, acc, 'bo', label='Training acc')
plt.plot(epochs, val_acc, 'b', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.show()
```

The dots are the training loss and accuracy, while the solid lines are the validation loss and accuracy. Note that your own results may vary slightly due to a different random initialization of your network.

As you can see, the training loss decreases with every epoch and the training accuracy increases with every epoch. That's what you would expect when running gradient descent optimization -- the quantity you are trying to minimize should get lower with every iteration. But that isn't the case for the validation loss and accuracy: they seem to peak at the fourth epoch. This is an example of what we were warning against earlier: a model that performs better on the training data isn't necessarily a model that will do better on data it has never seen before. In precise terms, what you are seeing is "overfitting": after the second epoch, we are over-optimizing on the training data, and we ended up learning representations that are specific to the training data and do not generalize to data outside of the training set.

In this case, to prevent overfitting, we could simply stop training after three epochs. In general, there is a range of techniques you can leverage to mitigate overfitting, which we will cover in the next chapter.

Let's train a new network from scratch for four epochs, then evaluate it on our test data:

```
In [70]: model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

model.compile(optimizer='rmsprop',
```

```

loss='binary_crossentropy',
metrics=['accuracy'])

model.fit(x_train, y_train, epochs=4, batch_size=512)
results = model.evaluate(x_test, y_test)

```

```

Epoch 1/4
49/49 ————— 1s 3ms/step - accuracy: 0.8014 - loss: 0.5121
Epoch 2/4
49/49 ————— 0s 3ms/step - accuracy: 0.8972 - loss: 0.3033
Epoch 3/4
49/49 ————— 0s 3ms/step - accuracy: 0.9184 - loss: 0.2314
Epoch 4/4
49/49 ————— 0s 4ms/step - accuracy: 0.9304 - loss: 0.1944
782/782 ————— 1s 747us/step - accuracy: 0.8874 - loss: 0.2796

```

In [71]: results

Out[71]: [0.2796470820903778, 0.8873599767684937]

Our fairly naive approach achieves an accuracy of 88%. With state-of-the-art approaches, one should be able to get close to 95%.

Using a trained network to generate predictions on new data

After having trained a network, you will want to use it in a practical setting. You can generate the likelihood of reviews being positive by using the `predict` method:

In [72]: model.predict(x_test)

```
782/782 ————— 0s 545us/step
```

```

Out[72]: array([[0.18157227],
               [0.99957675],
               [0.9370798 ],
               ...,
               [0.11911801],
               [0.09573828],
               [0.66813624]], shape=(25000, 1), dtype=float32)

```

As you can see, the network is very confident for some samples (0.99 or more, or 0.01 or less) but less confident for others (0.6, 0.4).

Further experiments

- We were using 2 hidden layers. Try to use 1 or 3 hidden layers and see how it affects validation and test accuracy.
- Try to use layers with more hidden units or less hidden units: 32 units, 64 units...
- Try to use the `mse` loss function instead of `binary_crossentropy`.
- Try to use the `tanh` activation (an activation that was popular in the early days of neural networks) instead of `relu`.

These experiments will help convince you that the architecture choices we have made are all fairly reasonable, although they can still be improved!

Conclusions

Here's what you should take away from this example:

- There's usually quite a bit of preprocessing you need to do on your raw data in order to be able to feed it -- as tensors -- into a neural

network. In the case of sequences of words, they can be encoded as binary vectors -- but there are other encoding options too.

- Stacks of `Dense` layers with `relu` activations can solve a wide range of problems (including sentiment classification), and you will

likely use them frequently.

- In a binary classification problem (two output classes), your network should end with a `Dense` layer with 1 unit and a `sigmoid` activation,

i.e. the output of your network should be a scalar between 0 and 1, encoding a probability.

- With such a scalar sigmoid output, on a binary classification problem, the loss function you should use is `binary_crossentropy`.
- The `rmsprop` optimizer is generally a good enough choice of optimizer, whatever your problem. That's one less thing for you to worry

about.

- As they get better on their training data, neural networks eventually start *overfitting* and end up obtaining increasingly worse results on data

never-seen-before. Make sure to always monitor performance on data that is outside of the training set.